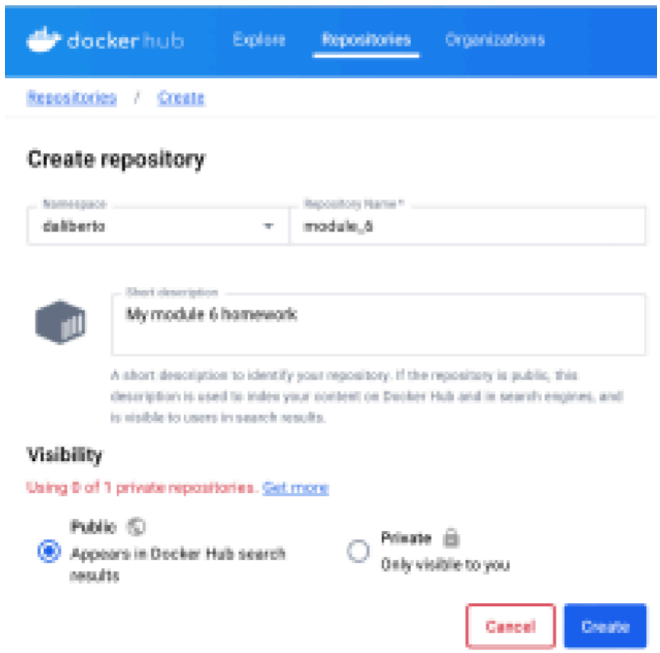
Module 6 - Deploy Anywhere Assignment

jhu.instructure.com/courses/112002/assignments/1235888



- Due Tuesday by 11:59pm
- Points 100
- Submitting a file upload

Introduction

In this assignment, you will refactor your SQL + Flask project into a microservice application. You will separate the web application from the database and introduce a background worker.

Why this design? Decoupling the web tier from data-modifying work improves reliability, scalability, and security: the web app remains fast and stateless, while a worker pool can scale independently, handle retries, and apply changes safely. All long-running or write operations flow through **RabbitMQ,** so you gain buffering and backpressure (no request timeouts under load), durable at-least-once delivery with idempotent handlers, and clearer operational visibility (queues, rates, dead-lettering). This mirrors industry patterns (producer/consumer, least-privilege data access) and reinforces software-assurance practices (parameterized SQL, controlled write paths). The stack is orchestrated via **Docker Compose,** so every component's configuration is explicit and reproducible on a clean machine—supporting consistent builds, easier testing, and deployment parity across student environments.

Skills: Containerization, Docker Images, Dockerfiles, Docker Hub, RabbitMQ (AMQP), Messaging Queues, Asynchronous Task Processing.

Assignment Overview

In this assignment, you will:

1. **Install Docker** (Desktop/Engine) and the **Docker Compose** plugin on your local machine.
2. **Scaffold a multi-service project** with four components: **web** (Flask), **worker** (Python), **db** (PostgreSQL), and **RabbitMQ** (message broker).
3. **Write Dockerfiles** for `web/` and `worker/` (pin dependencies, run as non-root).
4. **Author a** `docker-compose.yml` that defines services, a private network, health checks, and a named volume for Postgres.
   Move long-running/data-modifying work off the web tier by implementing a RabbitMQ **publisher** in `web/` (e.g., `scrape_new_data`, `recompute_analytics`) and a **consumer** in `worker/` (durable queue, acks, `prefetch=1`).
5. **Initialize or migrate the database schema** (reuse your Module 3 tables and any summaries/materialized views) and perform all SQL with **parameterized/composed queries**.
6. **Run and verify the stack** with `docker compose up --build`; exercise both tasks and confirm the UI updates after the worker processes messages; capture required screenshots.
7. **Tag and push images to Docker Hub** (or GHCR) for both `web/` and `worker/`, and document run instructions in your `README`.

After this assignment, you will have a **production-like microservice version** of your Module 3 project that **runs anywhere via Docker Compose, decouples** the web app from data-modifying work using **RabbitMQ**, is **reproducible from a clean machine**, and whose service images are **accessible/pullable from Docker Hub**.

1. Installing Docker

Installing Docker will look different dependent upon your available System or Windows version. However, it is a relatively tried and true process. Install instructions are available on Docker's website. For this assignment, we will specifically need the Docker CLI tool, and I have followed the Ubuntu-specific steps to install it. The steps may be different for you; however, confirmation that Docker is running correctly is nearly universal and looks like this:

2. Project Scaffold – Multi-Service Setup (web, worker, db, RabbitMQ)

Create a reproducible, containerized microservice baseline so all subsequent features (messaging, analytics recompute, security checks) build on a clean, consistent structure.

1. Pull your /module_5 codebase into a new folder called /module_6
2. Check that your unit test, lint, etc still passes
3. Create the following structure (move your `.github/workflows` and `.gitignore` content up to being directly under the repo base (i.e. inline with your main folder home and all your module_X folders:

```
module_6/
     docker-compose.yml # Non-empty: defines db, RabbitMQ, web, worker,
named volume, health checks
     setup.py
     README.md # run instructions, ports, registry links
     docs/
     tests/
     src/web/
          Dockerfile
          requirements.txt
          run.py # Flask entrypoint (bind 0.0.0.0:8080)
          publisher.py # RabbitMQ publisher (durable exchange/queue,
delivery_mode=2)
          app/
     src/worker/
          Dockerfile
          requirements.txt
          consumer.py # RabbitMQ consumer (acks, prefetch=1)
          etl/ # your data management / scraper code lives here
               incremental_scraper.py
               query_data.py
     src/db/
          load_data.py # JSON -> SQL loader
     src/data/
          applicant_data.json # Our LLM-cleaned applicant data
```

4. Add to your web and worker Dockerfiles:

web/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
USER 1000
CMD ["python", "run.py"]
```

worker/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
USER 1000
CMD ["python", "consumer.py"]
```

Shall

- **SHALL** create a `module_6/` folder with separate `web/` and `worker/` services.
- **SHALL** provide a `docker-compose.yml` that defines **four services**: `web` (Flask), `worker` (Python), `db` (PostgreSQL), and `RabbitMQ` (message broker).
- **SHALL** include a private network (Compose default) and a **named volume** for your `data/` folder.
- **SHALL** run `web/` on **port 8080**.

Should

- **SHOULD** run containers as a non-root user in both Dockerfiles.
- **SHOULD** include an initialization script for DB schema (e.g., `db/init.sql` or migrations).

Make sure to test that each individual container (web and worker) are able to run.

3. Write docker-compose.yml

Next, we need to create a clean, reproducible execution environment and a working multi-service stack that starts up with `Docker Compose`. This sets the stage for adding messaging and DB logic in the next step.

At the repo root, create a compose file (saved as `docker-compose.yml` that defines **db, RabbitMQ, web, and worker**, with healthchecks and a named volume for Postgres. Mount the `data/` directory read-only for the worker so it can load `applicant_data.json`.

It should look something like this (not identical, but close – may vary slightly based on your system but this should give you a strong first template)

Test that this file is able to start all containers.

4. Start Setting Things Up

1. Use your `load_data.py` file to initialize the empty PostgreSQL data. Fill the database with your data (now stored in data/applicant_data.json). Now, when you run `Docker Compose`, the PostgreSQL container should be automatically populated with data.
2. Now, we need to fill `publisher.py` to help us make our buttons "send messages"/make calls without needing to wait/remain on-page. We have our existing buttons. Ideally, on button click, our code will publish a task to RabbitMQ and immediately return HTTP 202 with a 'request queued' banner. Your `publisher.py` must include two functions:

   - `def _open_channel():`
   - `def publish_task(kind: str, payload: dict | None = None, headers: dict | None = None) -> None:`

   The following describes the contents of these functions:

   `_open_channel() → (conn, ch)`

   - **Reads:** `RABBITMQ_URL` from the environment.
   - **Creates** a connection (`BlockingConnection`) and channel.
   - **Declares** (idempotent):
     - a **durable direct exchange** named `tasks`
     - a **durable queue** named `tasks_q`
     - a **binding** with routing key `tasks`
   - **Returns:** the (`connection, channel`) pair.
   - **Notes:**
     - Declarations must be **durable** so messages/queues survive broker restarts.
     - You may enable **publisher confirms** on the channel (optional, recommended).

    - Do **not** leave connections open: callers will close them.

Should look approximately like this:

```
def _open_channel():
    url = os.environ["RABBITMQ_URL"]
    params = pika.URLParameters(url)
    conn = pika.BlockingConnection(params)
    ch = conn.channel()
    # Durable exchange & queue; bind once per process (idempotent)
    ch.exchange_declare(exchange=EXCHANGE, exchange_type="direct",
durable=True)
    ch.queue_declare(queue=QUEUE, durable=True)
    ch.queue_bind(exchange=EXCHANGE, queue=QUEUE,
routing_key=ROUTING_KEY)
return conn, ch
```

`publish_task(kind: str, payload: dict | None = None, headers: dict | None = None) -> None`

- **Builds** a compact JSON message with keys:
    - `kind` (e.g., `"scrape_new_data"`, `"recompute_analytics"`)
    - `ts` (UTC ISO timestamp)
    - `payload` (defaults to `{}`)
- **Publishes** to exchange `tasks` with routing key `tasks`.
  Sets `delivery_mode=2` (persistent message).
- **Closes** the connection in a `finally` block.
- **Error behavior**: raise an exception on failure so the Flask endpoint can return **503**. (Don't swallow
  errors.)

Should look approximately like this:

```
def publish_task(kind: str, payload: dict | None = None, headers: dict
| None = None) -> None

body = json.dumps(
    {"kind": kind, "ts": datetime.utcnow().isoformat(), "payload":
payload or {}},
    separators=(",", ":")
).encode("utf-8")
conn, ch = _open_channel()
```

```
try:
    ch.basic_publish(
        exchange=EXCHANGE,
        routing_key=ROUTING_KEY,
        body=body,
        properties=pika.BasicProperties(delivery_mode=2,
headers=headers or {}),
        mandatory=False,
    )
    # If using confirms:
    # if not ch.wait_for_confirms():
    # raise RuntimeError("Publish not confirmed")
finally:
    conn.close()
```

3. We need to connect our publisher functions to our existing Flask button functionality:

4. We need to connect our publisher functions to our existing Flask button functionality. So, under our empty file `worker/consumer.py`, we need to connect to RabbitMQ, consume from tasks_q with a basic_qos(prefetch_count=1), and ack after successful DB commit. We already have our incremental scraper file. All we need to do is write the RabbitMQ code around this system.
    1. Take your existing `incremental_scraper.py` (you may have named it something
       different) and rename it as `consumer.py`.
        - - Extend `db/load_data.py` to create a watermark table that ensures idempotent inserts.

          ```
          CREATE TABLE IF NOT EXISTS ingestion_watermarks (
                                source              TEXT PRIMARY
          KEY,
                                last_seen           TEXT,
             -- or TIMESTAMPTZ/INT; match your scraper's sort key
                                updated_at          TIMESTAMPTZ
          DEFAULT now()
          );
          ```
        - You **SHALL** read `last_seen` at the start of `handle_scrape_new_data` (or use
          `payload["since"]` if provided).

- You **SHALL** update `last_seen` to the max value seen in the batch **after** successful inserts.

2. Update your `consumer.py` to meet the following conditions:
   - **SHALL create** `worker/consumer.py`.
     Launch a long-running process that connects to RabbitMQ using `RABBITMQ_URL`.
   - **SHALL declare durable AMQP entities**.
     Declare a **durable direct exchange** `tasks`, a **durable queue** `tasks_q`, and **bind** them with routing key `tasks` (idempotent declarations).
   - **SHALL set backpressure.**
     Configure `basic_qos(prefetch_count=1)` so the worker processes **one message at a time.**
   - **SHALL consume and route by task kind**.
     Read each message's JSON; route by `kind` via a task map:
     `"scrape_new_data" → handle_scrape_new_data(conn, payload)`
     `"recompute_analytics" → handle_recompute_analytics(conn, payload)`
   - **SHALL open a DB transaction per message.**
     Use `DATABASE_URL` to connect to Postgres; start a transaction at handler entry and **commit only on success** (otherwise **rollback**).
   - **SHALL implement** `handle_scrape_new_data(conn, payload)`.
     Use existing scraper(s) to fetch **only records newer than a watermark**; normalize to the `applicants` schema; **batch-insert with parameterized SQL** and **idempotence** (`ON CONFLICT (ID_KEY) DO NOTHING` or equivalent); **advance the watermark** (in `ingestion_watermarks`) to the **max seen after commit**.
   - **SHALL implement** `handle_recompute_analytics(conn, payload)`.
     Recompute summaries/materialized views used by the UI (e.g., refresh a view) within the same per-message transaction; **commit on success**.
   - **SHALL acknowledge only after commit**.
     `basic_ack` **after** a successful commit; on any handler error, **rollback** and `basic_nack(requeue=false)` (no infinite retry loops).

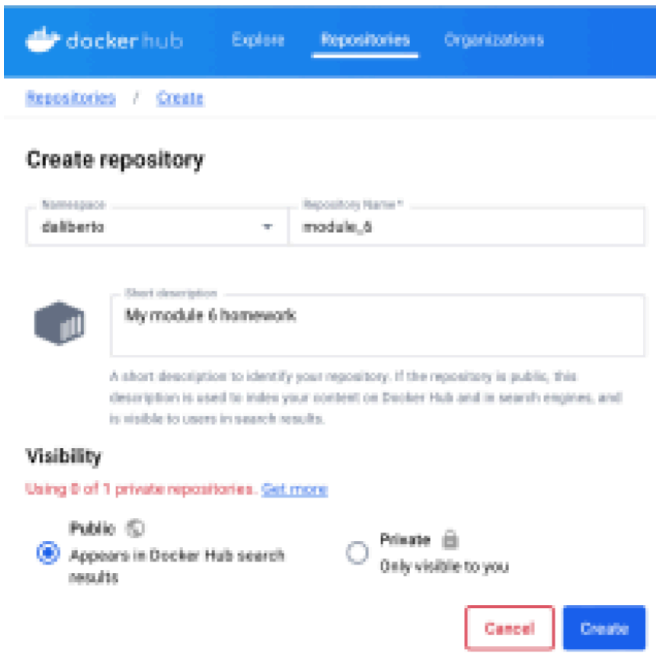
5. Build & Run

Execute `docker compose up --build`. Visit http://localhost:8080 for the app and http://localhost:15672 for RMQ (guest/guest, dev only). Trigger both tasks (buttons) and verify the UI updates after the worker processes them.

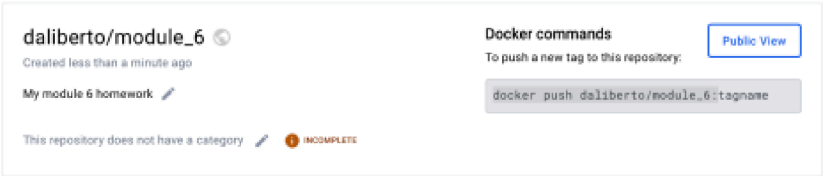
6. Upload your Docker Images to Dockerhub

Now that we have a viable set of containers, we want to make sure that it is accessible to others to pull into their own workspace. To do that, we need to put our image into a registry. To register your containers, follow the steps below:

1. Create a dockerhub account.
2. Create a public repository. You must name your repository `module_6`



3. Now we have a blank place to put our repository! We also have some instructions on how we ought to push images to this repository.



4. With this new location in mind, let's now create a copy of our image that includes the necessary repository (see docker command above), and we will tag our image **v1**.
   `ubuntu@ip-172-31-27-15:-/module_6$ sudo docker build -t daliberto/module_6:v1`
5. Now, log in to your docker account on your computer by using your dockerhub credentials:

```
ubuntu@ip-172-31-27-15:~/module_6$ sudo docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don
't have a Docker ID, head over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT g
rants better security and is required for organizations using SSO. Learn more at https://docs.d
ocker.com/go/access-tokens/

Username: daliberto
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

Login Succeeded
```

6. Finally, we can upload like so:
7. We can validate that our push was successful by checking dockerhub:

7. Deliverables

1. Submit a zipped copy of your `module_6` code to Canvas
2. Make your GitHub repo private again, and please submit the link
3. Push your `module_6` folder to GitHub
    1. Ensure a 10/10 lint
    2. Include a pdf that has a screenshot of your running website, RabbitMQ
    3. Include a link to your public `module_6` dockerhub repository.
    4. Ensure 100% unit test coverage + working tests (moving the lint github actions and pytest github actions to be compliant with module_6; all other github actions can remain module_5 driven).

You are not required to update your sphinx documentation. You are not required to submit any screenshots from prior weeks.

Please submit to both canvas and commit to your private GitHub!

Please let us know if you have any questions via Teams or email!